

Projeto Vaccai

Documentação Técnica e Modelagem Matemática

RUMENICK BRANDI SIGOLO

21 de fevereiro de 2026

Resumo

Este documento centraliza toda a parte matemática e os algoritmos utilizados no motor de processamento de áudio (DSP) do Projeto Vaccai. O foco principal é a análise de voz monofônica em tempo real.

O texto passa por todo o processo atual: desde o tratamento inicial do sinal e a detecção de frequência fundamental com o algoritmo YIN. Além disso, aborda a quantização para MIDI e o uso de Modelos Ocultos de Markov (HMM) para identificar a tonalidade e o modo da música. Sempre que novas lógicas ou funcionalidades forem adicionadas ao projeto, este documento também será atualizado para refletir a nova modelagem.

1. INTRODUÇÃO

O Projeto Vaccai é um sistema de processamento de áudio em tempo real voltado para o treinamento vocal. O principal objetivo do motor de DSP (*Digital Signal Processing*) é capturar a frequência fundamental (f_0) da voz com alta precisão e praticamente sem atraso (baixa latência), convertendo tudo para notas musicais para analisar a afinação e a tonalidade.

Esta documentação funciona como a base teórica e matemática oficial do *backend*. À medida que o código evoluir e novas soluções forem integradas, a modelagem por trás delas será continuamente adicionada aqui.

1.1 Justificativa da Abordagem no Domínio do Tempo

Para a detecção de f_0 , a escolha foi usar algoritmos no domínio do tempo (autocorrelação via YIN) em vez das clássicas abordagens espectrais via FFT. O motivo disso é o *trade-off* fundamental entre resolução espectral e temporal, limitado pelo princípio de Gabor-Heisenberg¹.

Em métodos de transformada discreta, a resolução de frequência Δf (a largura do *bin*) é inversamente proporcional ao tamanho da janela de tempo T_w :

$$\Delta f = \frac{f_s}{N} = \frac{1}{T_w} \quad (1)$$

Onde f_s é a taxa de amostragem e N o número de amostras. No caso do canto, a precisão nas frequências graves é crítica. Por exemplo, a diferença entre as notas A_2 (110 Hz) e $A\#_2$ (116.5 Hz) exige uma separação de apenas ≈ 6.5 Hz. Para conseguir resolver essa diferença direto na FFT, o tamanho mínimo da janela teria que ser:

¹O Princípio da Incerteza de Gabor estabelece que não dá para localizar um sinal com precisão perfeita no tempo e na frequência ao mesmo tempo. O produto das incertezas no tempo (Δt) e na frequência (Δf) tem um limite inferior: $\Delta t \cdot \Delta f \geq C$, onde a constante C depende de como se define a largura de banda.

$$T_w \approx \frac{1}{6.5 \text{ Hz}} \approx 153 \text{ ms} \quad (2)$$

Uma latência de 153 ms, somada ao tempo de computação e ao *buffer* de áudio, criaria um atraso perceptível. Isso inviabilizaria o *feedback* visual instantâneo, que é o objetivo do Projeto Vaccai.

Por outro lado, métodos no domínio do tempo funcionam medindo a periodicidade do sinal (atraso τ). Isso permite atingir uma precisão de frações de Hertz usando janelas bem menores (geralmente 2 a 3 períodos da fundamental), o que garante a resposta rápida do sistema.

2. MODELO DE SINAL E PRÉ-PROCESSAMENTO

Para manter a baixa latência justificada na seção anterior, o sistema trabalha diretamente no fluxo de áudio bruto. A ideia é evitar o uso de janelas de acumulação muito extensas, o que criaria um atraso perceptível. Contudo, a eficiência dos algoritmos no domínio do tempo depende muito da qualidade do sinal de entrada.

Considerando $x[n]$ o sinal de áudio discreto com taxa de amostragem f_s (amostras por segundo), onde n é o índice da amostra no tempo, o processamento ocorre em janelas (*buffers*) de tamanho fixo N . Esse tamanho é dimensionado justamente para garantir a resposta em tempo real.

Na análise vocal, o sinal $x[n]$ não é contínuo; ele é intercalado por pausas de silêncio e ruídos não-vocais (como respiração, som ambiente ou consoantes fricativas). O pré-processamento tem o papel crítico de “limpar” esse sinal e decidir, de forma determinística, quando o algoritmo de detecção de *pitch* deve atuar. Isso evita o processamento de ruídos que gerariam resultados musicais espúrios (notas erradas). A lógica completa desse condicionamento e decisão de fluxo está detalhada no **Algoritmo 1**, no Apêndice A.

2.1 Remoção de Componente Contínua (DC Offset)

Sinais de voz capturados por microfones comuns costumam ter um deslocamento de tensão vertical, conhecido como *DC Bias*. Isso acontece por conta de pequenas imperfeições no hardware de conversão analógico-digital, fazendo com que a onda não oscile exatamente em torno do zero.

Esse deslocamento atrapalha a análise de periodicidade, já que altera os valores no cálculo dos produtos $x[n] \cdot x[n + \tau]$. A solução para corrigir isso é centralizar o sinal subtraindo a sua média:

$$\mu_x = \frac{1}{N} \sum_{n=0}^{N-1} x[n] \quad (3)$$

$$x'[n] = x[n] - \mu_x \quad (4)$$

Essa operação traz a simetria da onda de volta, o que é fundamental para conseguir extrair o *pitch* com precisão.

2.2 Discriminação Híbrida de Voz e Silêncio (Histerese Dupla)

A detecção de atividade vocal combina a energia do sinal (RMS) com a pureza harmônica para decidir de forma assertiva sobre a presença de voz. Para medir a energia, calcula-se a amplitude *Root Mean Square* para cada janela:

$$A_{rms}[t] = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} (x'[n])^2} \quad (5)$$

Uma melhoria importante é a capacidade de diferenciar vogais de ruídos (como /s/ ou respiração) usando a Taxa de Cruzamento por Zero (ZCR):

$$ZCR[t] = \frac{1}{N-1} \sum_{n=1}^{N-1} \mathbb{I}(x'[n] \cdot x'[n-1] < 0) \quad (6)$$

Para decidir o estado de processamento $S_t \in \{0, 1\}$ (onde 1 é "Voz" e 0 é "Silêncio"), um comparador simples oscilaria rapidamente gerando *chattering*. O sistema resolve isso implementando uma **histerese dupla**, que avalia simultaneamente amplitude e ZCR:

$$S_t = \begin{cases} 1 & \text{se } A_{rms}[t] > \eta_{open} \wedge ZCR[t] < \zeta_{open} \\ 0 & \text{se } A_{rms}[t] < \eta_{close} \vee ZCR[t] > \zeta_{close} \\ S_{t-1} & \text{caso contrário} \end{cases} \quad (7)$$

Os parâmetros foram rigorosamente calibrados para garantir responsividade:

- **Limiares de Abertura** ($\eta_{open} = 0.003$, $\zeta_{open} = 0.20$): O áudio deve ter volume suficiente e cruzar o zero poucas vezes (indicando um som vozeado limpo) para iniciar a análise.
- **Limiares de Fechamento** ($\eta_{close} = 0.001$, $\zeta_{close} = 0.30$): Se o áudio cair abaixo do piso de ruído ou se tornar excessivamente sibilante (ZCR alto), o motor de processamento entra em pausa.

Essa "zona morta" funciona como uma inércia digital, garantindo que o *feedback* visual tenha brechas limpas e não trema durante finais de frase orgânicos.

3. DETECÇÃO DE FREQUÊNCIA FUNDAMENTAL (ALGORITMO YIN)

Quando o pré-processamento confirma que existe atividade vocal estável ($S_t = 1$), o sistema avança para a extração da frequência fundamental f_0 . Para essa tarefa, utiliza-se o **Algoritmo YIN** [1], escolhido por ser significativamente mais robusto que métodos simples de contagem de cruzamentos por zero ou correlação padrão. A estrutura completa dessa implementação encontra-se no **Algoritmo 2**, no Apêndice A.

3.1 Função de Diferença e Normalização Cumulativa

O núcleo matemático do YIN opera de forma inversa à correlação: em vez de buscar a máxima semelhança, ele busca minimizar a diferença entre o sinal condicionado $x'[n]$ e suas versões atrasadas. A função de diferença quadrática é definida como:

$$d_t(\tau) = \sum_{j=1}^W (x'[j] - x'[j + \tau])^2 \quad (8)$$

Para evitar o erro de "atraso zero" e impedir que o algoritmo escolha harmônicos superiores por engano, aplica-se a Normalização pela Média Cumulativa (CMNDF). Essa operação cria a função $d'_t(\tau)$, que penaliza vales profundos em atrasos curtos caso eles não se sustentem na média, reduzindo drasticamente os erros de oitava:

$$d'_t(\tau) = \begin{cases} 1 & \text{se } \tau = 0 \\ d_t(\tau) \cdot \left[\frac{1}{\tau} \sum_{j=1}^{\tau} d_t(j) \right]^{-1} & \text{se } \tau > 0 \end{cases} \quad (9)$$

3.2 Seleção de Limiar (Busca Híbrida)

A literatura original sugere um limiar absoluto e rigoroso em torno de 0.10 para evitar erros de oitava definitivamente. No entanto, para fins práticos e pedagógicos, o motor DSP do Projeto Vaccai implementa uma estratégia de **busca híbrida**.

Primeiro, o algoritmo procura o primeiro vale que cruze um limite de tolerância estrito de $\theta = 0.15$. Se a voz do estudante iniciante for muito "soprosa" (com muito ar vazando), a aperiodicidade será alta e esse limiar rigoroso pode não ser alcançado. Neste caso específico, o algoritmo realiza um *fallback* para o mínimo global encontrado na janela de atrasos.

O valor detectado só é totalmente rejeitado se o índice de clareza do sinal for inferior a 0.60 (o equivalente a uma taxa de aperiodicidade de 0.40). Isso assegura precisão de estúdio quando o som é limpo, sem punir ou ignorar a voz em desenvolvimento de um estudante que ainda carece de técnica vocal perfeita.

3.3 Refinamento Parabólico

A detecção discreta fica limitada a valores inteiros de amostras. Para frequências agudas, como um Dó Soprano ($C6 \approx 1047$ Hz), um erro de arredondamento de apenas 0.5 amostra já causaria um desvio de afinação próximo a **20 cents**. Isso é inaceitável para treino auditivo, já que o ouvido humano percebe diferenças menores que 5 cents².

Para superar essa limitação física da taxa de amostragem sem precisar recorrer a um *oversampling* custoso, aplica-se uma interpolação parabólica sobre o mínimo encontrado τ_{int} . O período refinado τ_{fino} é calculado pelo vértice da parábola ajustada aos vizinhos do mínimo na função CMNDF:

$$\tau_{fino} = \tau_{int} + \frac{d'[\tau_{int} - 1] - d'[\tau_{int} + 1]}{2(d'[\tau_{int} - 1] - 2d'[\tau_{int}] + d'[\tau_{int} + 1])} \quad (10)$$

A frequência final é dada por $f_0 = f_s / \tau_{fino}$, garantindo precisão sub-cent sem adicionar latência.

3.4 Cadeia de Filtragem e Estabilização Temporal

A frequência bruta $\hat{f}_{raw}[n]$ obtida pelo YIN pode sofrer com dois tipos de instabilidade: erros impulsivos (saltos bruscos de oitava) e pequenas flutuações rápidas (*jitter*). Para limpar isso sem comprometer a resposta em tempo real, o sistema usa uma cadeia de dois filtros: um não-linear seguido de um linear.

3.4.1 Estágio 1: Filtragem de Mediana (Rank-Order Filter)

Para eliminar erros grosseiros (*outliers*) mantendo as transições reais entre as notas, o sinal passa primeiro por um filtro de mediana com janela $L = 5$. Considerando o vetor de amostras recentes $W_n = [\hat{f}_{raw}[n], \dots, \hat{f}_{raw}[n - 4]]$, a saída é:

$$f_{med}[n] = \text{Mediana}(W_n) \quad (11)$$

A escolha de uma janela ímpar de tamanho 5 permite ignorar até dois *frames* errados consecutivos, introduzindo uma latência mínima de $(L - 1)/2 = 2$ amostras, o que é essencial para a responsividade.

²A sensibilidade auditiva à variação de altura em ouvintes treinados pode ser inferior a 5 cents (aprox. 0.3% da frequência) nas regiões centrais da fala e canto (Moore, B. C. J., 2012).

3.4.2 Estágio 2: Suavização Exponencial (IIR)

Em seguida, o sinal $f_{med}[n]$ passa por um filtro IIR de primeira ordem, que atua como um integrador com perdas. A equação é:

$$f_{final}[n] = \alpha \cdot f_{med}[n] + (1 - \alpha) \cdot f_{final}[n - 1] \quad (12)$$

O valor de α foi definido em 0.50. Esse é o ponto de equilíbrio: ele garante que a visualização da nota tenha firmeza e não fique tremendo na tela, mas sem deixar o sistema lento. Assim, ainda é possível capturar variações rápidas da voz.

4. MAPEAMENTO MUSICAL (CONVERSÃO MIDI)

Depois de obter a frequência fundamental $\hat{f}_0$ (em Hertz) pelo algoritmo YIN, o sistema converte esse valor para o espaço MIDI. Essa transformação é necessária para linearizar a percepção de altura. Na prática, isso garante que intervalos musicais iguais correspondam a distâncias numéricas constantes, não importa se a nota está numa oitava grave ou aguda.

4.1 Modelagem Matemática

O sistema usa como base o padrão internacional ISO 16:1975 [3], fixando o Lá central (A_4) em $\lambda = 440$ Hz. Isso garante compatibilidade com instrumentos ocidentais e serve de âncora para o índice MIDI $\mu = 69$. A função de mapeamento $\mathcal{M} : \mathbb{R}^+ \rightarrow \mathbb{R}$ é definida pela equação logarítmica:

$$m = 69 + 12 \cdot \log_2 \left(\frac{\hat{f}_0}{440} \right) \quad (13)$$

O resultado m é um número real que guarda, ao mesmo tempo, a altura da nota e a precisão da afinação. Para saber qual é a nota musical exata n na escala temperada, basta arredondar para o inteiro mais próximo:

$$n = \lfloor m \rceil \quad (14)$$

4.2 Quantificação do Erro de Afinação (Biofeedback)

A parte decimal que sobra define o desvio de afinação, chamado de $\delta = m - n$. No treino vocal, visualizar esse desvio é fundamental, já que a voz humana é um instrumento de altura contínua (não tem teclas fixas como um piano).

O sistema normaliza esse valor para a unidade de *cents* (centésimos de semitom) usando a relação:

$$c = 100 \cdot (m - n) \quad (15)$$

O resultado c fica no intervalo $[-50, +50]$. Esse dado alimenta a interface em tempo real, mostrando se o estudante está desafinando para baixo (bemol) ou para cima (sustenido). Assim, ele consegue corrigir a emissão vocal para chegar no centro tonal n , treinando o ouvido e o controle vocal.

5. ANÁLISE DE ESTABILIDADE VOCAL

A seção anterior mostrou como obter a nota MIDI instantânea $m[t]$. Só que ter uma boa técnica vocal não é apenas acertar a afinação por um instante; é ter a capacidade de sustentar a emissão com consistência, evitando oscilações involuntárias (*jitter* de altura) ou instabilidade por falta de apoio respiratório.

Para medir isso em tempo real, implementou-se uma métrica de estabilidade baseada na dispersão estatística de curto prazo.

5.1 Modelagem Estatística (Janela Deslizante)

O sistema mantém um *buffer* circular com as últimas K amostras de notas MIDI detectadas. O tamanho da janela foi definido em $K = 15$ quadros. Esse valor foi escolhido para oferecer um equilíbrio entre suavizar micro-flutuações irrelevantes e manter a reatividade para detectar tremolos reais.

Seja $\mathcal{M} = \{m_1, m_2, \dots, m_K\}$ o conjunto de valores na janela atual. O sistema calcula a média aritmética μ e a variância σ^2 :

$$\mu = \frac{1}{K} \sum_{i=1}^K m_i, \quad \sigma^2 = \frac{1}{K} \sum_{i=1}^K (m_i - \mu)^2 \quad (16)$$

O desvio padrão da amostra recente é dado por $\sigma = \sqrt{\sigma^2}$.

5.2 Métrica de Pontuação Normalizada

A pontuação de estabilidade $S \in [0, 1]$ é calculada normalizando o desvio padrão em relação a um limite de tolerância σ_{lim} .

No código, fixou-se $\sigma_{lim} = 0.5$ semitons. Esse valor representa o limite entre um vibrato artístico controlado e uma instabilidade indesejada. A função de pontuação final é:

$$S = 1.0 - \min\left(\frac{\sigma}{\sigma_{lim}}, 1.0\right) \quad (17)$$

Dessa forma, uma emissão perfeitamente reta ("lisa") resulta em $S \approx 1.0$, enquanto oscilações que passam de meio semitom de amplitude derrubam a pontuação em direção a 0.0. Isso fornece um *biofeedback* visual imediato sobre o controle de ar e a tensão laríngea do estudante.

6. INFERÊNCIA DE TONALIDADE (KEY DETECTION)

Na seção anterior, o sistema obteve uma sequência de notas ao longo do tempo. Só que, na música ocidental, o significado de uma nota depende do contexto harmônico (a Tonalidade). Diferente da análise polifônica, onde os acordes deixam o tom óbvio, o Projeto Vaccai trabalha com sinal monofônico (apenas a melodia). Isso exige que o sistema descubra o contexto global a partir de uma linha melódica que, muitas vezes, contém erros de execução.

Para resolver essa ambiguidade das notas isoladas, utiliza-se um Modelo Oculto de Markov (HMM). A abordagem adapta o método de Müller [4] para priorizar a memória temporal e a estabilidade. A implementação detalhada está no **Algoritmo 3** (Apêndice A).

6.1 Espaço de Estados e Observação

O espaço de estados S é formado pelas 24 tonalidades (12 maiores e 12 menores). Seja $q_t \in S$ a tonalidade oculta no instante t . A variável observável o_t não é a frequência absoluta, mas sim a **classe de altura** (ou *chroma*), derivada da nota MIDI m calculada na Seção 4:

$$o_t = (m \bmod 12) \in [0, 12) \quad (18)$$

Essa operação de módulo torna o sistema "cego" à oitava. Ou seja, ele entende que um Dó central e um Dó agudo são a mesma classe de nota, permitindo detectar a tonalidade independente da voz ser grave (Baixo) ou aguda (Soprano).

6.2 Probabilidade de Emissão (Modelo Fuzzy)

A probabilidade de emissão $P(o_t|q_t = k)$ calcula a chance de se observar o chroma o_t dado que a tonalidade atual é k . Para lidar com a afinação da voz humana (que tem vibratos e micro-desvios), o sistema não usa uma lógica binária ("pertence ou não pertence à escala"). Em vez disso, usa uma abordagem *Fuzzy Gaussian*.

Seja $d(o_t, S_k)$ a menor distância circular no relógio cromático entre a nota observada o_t e as notas da escala S_k . A função é:

$$E(o_t|k) = e^{-\frac{d(o_t, S_k)^2}{2\sigma^2}} + \epsilon \quad (19)$$

Justificativa dos Parâmetros:

- $\epsilon = 0.01$: Um valor mínimo de probabilidade para evitar erros numéricos (divisão por zero) em notas fora da escala.
- $\sigma = 0.40$: Esse valor define a tolerância. Foi estabelecido que uma nota com erro de afinação de ± 0.4 semitons ainda conta com $\approx 60\%$ de relevância para a tonalidade correta. Isso permite que o sistema "perdoe" execuções imperfeitas, mantendo o contexto harmônico estável mesmo que o estudante desafine um pouco.

6.3 Dinâmica de Transição e Inércia Tonal

A matriz de transição A define a probabilidade da música mudar de tom. Como modulações bruscas são raras em exercícios vocais, a matriz foi construída para favorecer a continuidade (auto-transição), criando uma espécie de "inércia".

As chances de transição para outros tons $A_{i,j}$ são baseadas na distância harmônica no **Círculo de Quintas**³. Isso permite modulações suaves (ex: Dó Maior $\rightarrow$ Sol Maior), mas penaliza saltos distantes e estranhos (ex: Dó Maior $\rightarrow$ F# Maior).

6.4 Atualização Causal Otimizada

Em sistemas tradicionais de análise musical **offline**, algoritmos como o de Viterbi são frequentemente usados, pois analisam todo o áudio futuro para deduzir as escolhas corretas do passado. O motor DSP do Vaccai é estritamente de tempo real (causal), o que inviabiliza olhar para quadros futuros.

Para compensar isso e manter a estabilidade, o algoritmo considera a transição a partir da tonalidade mais provável do instante anterior (k_{best}) com um peso de inércia altíssimo. Seja $\mathbf{P}_t$ o vetor de probabilidades para as 24 tonalidades:

$$\mathbf{P}_t[k] \propto E(o_t|k) \cdot (\alpha \cdot \mathbf{P}_{t-1}[k] + (1 - \alpha) \cdot A_{k_{best},k}) \quad (20)$$

Onde $\alpha = 0.9$ é o fator de inércia. Essa imposição causal (90% de memória sobre 10% de transição) reduz o custo de processamento a zero atraso perceptível e evita modulações errôneas originadas por notas isoladas fora do tom.

7. CONCLUSÃO E TRABALHOS FUTUROS

A base matemática e os algoritmos apresentados aqui formam o coração do motor de processamento de sinais do Projeto Vaccai. A arquitetura foi pensada com um foco principal: garantir que a interação seja realmente em tempo real. Ao escolher algoritmos no domínio do tempo (YIN) e otimizar o fluxo

³O Círculo de Quintas organiza as 12 tonalidades. Nele, tonalidades vizinhas estão a um intervalo de quinta justa (7 semitons) e compartilham quase as mesmas notas.

de processamento, foi possível criar um sistema que dá *feedback* visual instantâneo. Ele mostra a afinação com uma latência que ninguém percebe, rodando bem até em dispositivos com recursos limitados.

Este documento define a "camada de performance": as rotinas que precisam rodar obrigatoriamente ao mesmo tempo que a pessoa está cantando.

7.1 Trabalhos Futuros e Viabilidade de Expansão

O cenário atual de tecnologia para áudio e inteligência artificial trouxe muitas ferramentas novas que podem ser extremamente úteis no aprendizado de canto. Como o Projeto Vaccai é uma iniciativa *open source*, nascida de uma demanda pessoal por métricas claras de estudo, a ideia é integrar essas inovações gradualmente, tanto pelo desafio técnico quanto para oferecer mais recursos para a comunidade.

Algumas das possibilidades que estão no radar incluem:

- **Avaliação de Pronúncia:** Usar reconhecimento de fala (ASR) para checar se o aluno está articulando a letra da música corretamente, e não apenas acertando a melodia.
- **Separação de Áudio (Stems):** Implementar redes neurais para remover a voz original de músicas comerciais, permitindo que o aluno pratique com instrumentais reais ("Karaokê Inteligente").
- **Correção via IA Generativa:** Utilizar modelos mais modernos para corrigir a afinação mantendo a naturalidade da voz, superando os métodos antigos que deixavam o som robotizado.

O grande desafio dessa expansão é puramente computacional. Rodar modelos de Deep Learning modernos exige hardware robusto (geralmente GPUs potentes). O obstáculo para as próximas versões será otimizar esses recursos pesados para que funcionem em computadores domésticos comuns, garantindo que o software continue acessível para quem não possui uma máquina de alto desempenho.

A. IMPLEMENTAÇÃO ALGORÍTMICA

Abaixo apresentamos os algoritmos fundamentais que implementam o modelo matemático descrito. O Algoritmo 1 detalha o condicionamento de sinal e a lógica de histerese. O Algoritmo 2 descreve a detecção de *pitch* via YIN com pós-filtragem. Por fim, o Algoritmo 3 descreve a inferência harmônica Fuzzy.

Algoritmo 1 Pré-processamento: Noise Gate com Histerese e ZCR

Entrada: Buffer de áudio bruto x de tamanho N , Estado anterior S_{prev}

Saída: Buffer centralizado x' , Novo Estado S

```

1:  $\mu \leftarrow \frac{1}{N} \sum x[i]$  ▷ Média DC
2:  $E \leftarrow 0$ ,  $crossings \leftarrow 0$ 
3: for  $i \leftarrow 0$  to  $N - 1$  do
4:    $x'[i] \leftarrow x[i] - \mu$  ▷ Remoção de Offset
5:    $E \leftarrow E + (x'[i])^2$ 
6:   if  $i > 0$  and  $\text{sgn}(x'[i]) \neq \text{sgn}(x'[i - 1])$  then
7:      $crossings \leftarrow crossings + 1$ 
8:   end if
9: end for
10:  $rms \leftarrow \sqrt{E/N}$ 
11:  $zcr \leftarrow crossings / (N - 1)$ 
12: Lógica de Histerese Dupla (Schmitt Trigger)
13: if  $rms > 0.003$  and  $zcr < 0.20$  then
14:    $S \leftarrow \text{True}$  ▷ Abre o Gate (Voz Limpa Detectada)
15: else if  $rms < 0.001$  or  $zcr > 0.30$  then
16:    $S \leftarrow \text{False}$  ▷ Fecha o Gate (Silêncio ou Ruído de Sopro)
17: else
18:    $S \leftarrow S_{prev}$  ▷ Mantém estado na Zona Morta
19: end if
20: if  $S$  is True then
21:   return True,  $x'$ 
22: else
23:   return False,  $\emptyset$ 
24: end if

```

Algoritmo 2 Estimativa de f_0 via Algoritmo YIN com Filtragem Híbrida**Entrada:** Sinal x' , Taxa f_s , Histórico f_{prev} **Saída:** Frequência Estabilizada f_{final}

```

1:  $\tau_{min} \leftarrow f_s/1000$ ,  $\tau_{max} \leftarrow f_s/50$ 
2: Inicializar vetor de diferença  $d[\tau]$  e CMNDF  $d'[\tau]$ 
3: Passo 1: Diferença Quadrática e CMNDF
4:  $d'[0] \leftarrow 1$ ,  $running\_sum \leftarrow 0$ 
5: for  $\tau \leftarrow 1$  to  $\tau_{max}$  do
6:    $d[\tau] \leftarrow \sum (x'[i] - x'[i + \tau])^2$ 
7:    $running\_sum \leftarrow running\_sum + d[\tau]$ 
8:    $d'[\tau] \leftarrow d[\tau] \cdot (\tau/running\_sum)$ 
9: end for
10: Passo 2: Busca Híbrida por Vale (Limiar + Fallback)
11:  $\hat{\tau} \leftarrow -1$ ,  $min\_val \leftarrow \infty$ 
12: for  $\tau \in [\tau_{min}, \tau_{max}]$  do
13:   if  $d'[\tau] < 0.15$  then ▷ Limiar Rigoroso
14:     while  $d'[\tau + 1] < d'[\tau]$  do ▷ Busca mínimo local
15:        $\tau \leftarrow \tau + 1$ 
16:     end while
17:      $\hat{\tau} \leftarrow \tau$ 
18:      $min\_val \leftarrow d'[\hat{\tau}]$ 
19:     Break
20:   end if
21:   if  $d'[\tau] < min\_val$  then ▷ Grava para o Fallback
22:      $min\_val \leftarrow d'[\tau]$ 
23:      $global\_min\_tau \leftarrow \tau$ 
24:   end if
25: end for
26: if  $\hat{\tau} == -1$  then ▷ Aplica Fallback se não bater 0.15
27:    $\hat{\tau} \leftarrow global\_min\_tau$ 
28: end if
29: Passo 3: Validação por Claridade
30:  $clarity \leftarrow 1.0 - min\_val$ 
31: if  $clarity < 0.60$  then ▷ Corte de pureza final (0.40 aperiodicidade)
32:   return 0
33: end if
34:  $f_{raw} \leftarrow f_s / \text{ParabolicInterpolation}(\hat{\tau})$ 
35: Passo 4: Filtragem Híbrida (Mediana + IIR)
36:  $f_{med} \leftarrow \text{MedianFilter}(f_{raw}, \text{buffer\_size} = 5)$ 
37:  $f_{final} \leftarrow 0.5 \cdot f_{med} + 0.5 \cdot f_{prev}$  ▷ Suavização IIR
38: return  $f_{final}$ 

```

Algoritmo 3 Detecção de Tonalidade Causal (HMM Gaussiano Fuzzy)**Entrada:** Nota MIDI n , Vetor de Probabilidades $\mathbf{P}_{prev}$ **Saída:** Tonalidade Vencedora K_{win}

```

1:  $c \leftarrow n \pmod{12}$  ▷ Nota cromática contínua
2:  $\mathbf{P}_{new} \leftarrow \text{zeros}(24)$ 
3:  $k_{best\_prev} \leftarrow \text{argmax}(\mathbf{P}_{prev})$ 
4:  $\sigma \leftarrow 0.40$  ▷ Parâmetro de tolerância
5: for  $k \leftarrow 0$  to 23 do
6:   Passo 1: Emissão Gaussiana
7:    $d_{min} \leftarrow \infty$ 
8:   for  $degree \in \text{ScaleDegrees}[k]$  do
9:      $d \leftarrow \text{CircularDistance}(c, degree)$ 
10:     $d_{min} \leftarrow \min(d_{min}, d)$ 
11:   end for
12:    $E \leftarrow e^{-(d_{min}^2/2\sigma^2)} + 0.01$ 
13:   Passo 2: Transição e Inércia Causal
14:    $T \leftarrow \text{TransitionMatrix}[k_{best\_prev}][k]$ 
15:    $\mathbf{P}_{new}[k] \leftarrow E \cdot (0.9 \cdot \mathbf{P}_{prev}[k] + 0.1 \cdot T)$ 
16: end for
17: Passo 3: Normalização
18:  $S \leftarrow \sum \mathbf{P}_{new}$ 
19: if  $S > 0$  then
20:    $\mathbf{P}_{prev} \leftarrow \mathbf{P}_{new} / S$  ▷ Atualiza e normaliza
21: else
22:    $\mathbf{P}_{prev} \leftarrow \text{Uniform}(24)$  ▷ Fallback
23: end if
24:  $K_{win} \leftarrow \text{argmax}(\mathbf{P}_{prev})$ 
25: return  $K_{win}$ 

```

REFERÊNCIAS

- [1] De Cheveigné, Alain; Kawahara, Hideki. “YIN, a fundamental frequency estimator for speech and music”. *The Journal of the Acoustical Society of America*, vol. 111, no. 4, pp. 1917-1930. 2002.
- [2] Moore, Brian C. J. *An Introduction to the Psychology of Hearing*. 6th Edition. Emerald Group Publishing Limited, 2012.
- [3] International Organization for Standardization. *ISO 16:1975 Acoustics — Standard tuning frequency (Standard musical pitch)*. 1975.
- [4] Müller, Meinard. *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Capítulo 5: Harmony Analysis. Springer, 2015. ISBN 978-3-319-21944-8.